Distributed systems, Cloud and AI

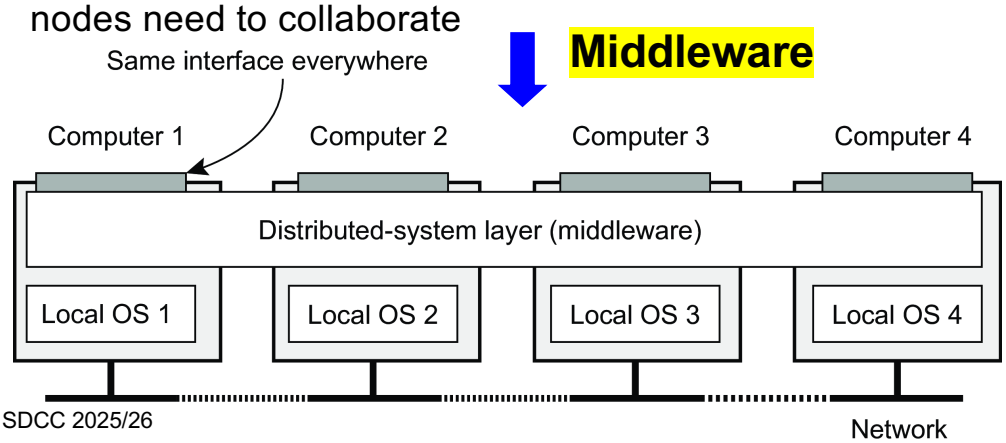
- AI has recently become practical as result of:
  - Distributed computing
  - Affordable cloud computing and storage costs
  - Examples: federated learning, distributed training of foundation models (huge computational and data storage needs)
- Distribute = to divide and dispense in portions
- A foremost strategy used in distributed computing you already know
  - Divide et impera: break larger (computational) problems down into numbers of smaller, interrelated, “manageable” pieces

Distributed system

- Multiple definitions of **distributed system (DS)**

[van Steen & Tanenbaum] A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system

  - Consists of autonomous computing elements (i.e., **nodes**), can be hardware devices (computer, phone, car, robot, …) or software processes
  - Users or applications perceive it as a **single system** (how?): nodes need to collaborate



Distributed system

[Lamport] A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable

- Emphasis on fault tolerance

- Who is Leslie Lamport?
  - Recipient of 2013 Turing award https://www.youtube.com/watch?v=pgWTmOyUjtM
  - His research contributions have laid the foundations of theory and practice of DS
    - Fundamental concepts such as causality, logical clocks and Byzantine failures
    - Algorithms to solve many fundamental problems in DS
  - Inventor of LaTeX

Why make a system distributed?

- Share resources
  - Resource = computing node, storage, data, service, …
- Lower costs
- Improve performance
  - e.g., get data from a nearby node rather than one halfway round the world
- Improve availability and reliability
  - even if one node fails, the system as a whole keeps functioning
- Improve security
- Solve bigger problems
  - e.g., huge amounts of data, can't fit on one machine
- Support Quality of Service (QoS) and Quality of Experience (QoE)

Why to study distributed systems

- Distributed systems are more complex than centralized ones
  - e.g., no global clock, group membership, …
- Building them is harder… and building them correct is even much harder

  Distributed systems need radically different software than centralized systems do (Tanenbaum)
- Managing, and, above all, testing them is difficult

Some distinguishing features of DS

- Concurrency
  - Many things happen “at the same time”
  - Centralized system: design choice
  - Distributed system: fact of life to be dealt with
- No global clock
  - Centralized system: use computer’s physical clock for synchronization
  - Distributed system: many physical clocks and not necessarily synchronized among them
- Independent and partial failures
  - Centralized system: fails completely
  - Distributed system: fails partially (i.e., only a part), often due to communication; hard (and in general impossible) to hide partial failures and their recovery

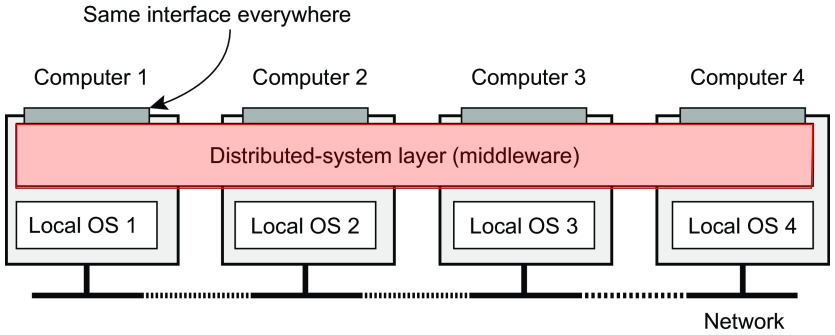
Challenges and design goals

- Challenges and goals associated with designing distributed systems
  1. **Heterogeneity**
  2. **Distribution transparency**
  3. **Openness**
  4. **Scalability**
  5. **Dependability**
  6. **Security**

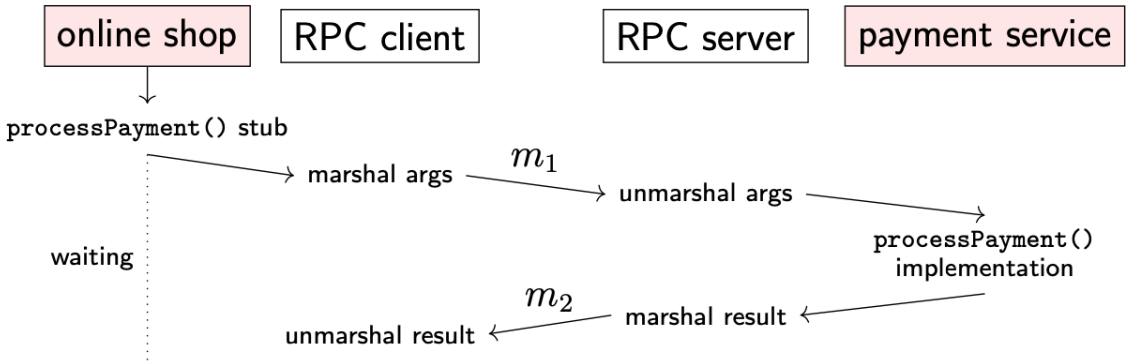
while improving performance and sustainability (energy efficiency, carbon-awareness), reducing monetary cost, etc.

Challenge 1: Heterogeneity

- Many sources of heterogeneity: network, hardware, operating system (OS), programming language, implementations by different developers
- How to address? **Middleware**: the "OS of a DS"
  - Sw layer placed on top of OS that provides a programming abstraction as well as masks heterogeneity
  - Contains commonly used components and functionalities (e.g., communication) thus avoiding developers to implement them again and from scratch



RPC: Behind the curtains



$m_1 =$

```
{
  "request": "processPayment",
  "card": {
    "number": "1234567887654321",
    "expiryDate": "10/2024",
    "CVC": "123"
  },
  "amount": 3.99,
  "currency": "GBP"
}
```

$m_2 =$

```
{
  "result": "success",
  "id": "XP61hHw2Rvo"
}
```


Challenge 2: Distribution transparency

- **Distribution transparency**: single coherent system where distribution of its *objects* (*resources* and *processes*) is *transparent* (i.e., invisible) to users and applications
- **Types** of distribution transparency (*ISO 10746*, Reference Model of Open Distributed Processing)

Access transparency

- Hide differences in data representation and how an object is accessed
  - e.g., use same mechanism for local or remote call

Location transparency

- Hide where an object is located
  - e.g., URL hides IP address

Relocation transparency

- Hide that an object may be moved to another location *while in use*

Challenge 2: Distribution transparency

Migration transparency

- Hide that an object may move to another location
  - e.g., communication between mobile phones

Replication transparency

- Hide that an object may be replicated
  - How? Same name for all replicas
  - Example: type command `dig www.youtube.com`
  - Require also location transparency

Concurrency transparency

- Hide that an object may be shared by several users
  - E.g.: concurrent accesses to DB table
  - Issue: leave shared object in a consistent state, e.g., by *locking* mechanisms

Failure transparency

- Hide failure and recovery of an object (Lamport's DS definition)

Degree of distribution transparency

- Aiming at *full* distribution transparency may be too much
  - We cannot always hide communication latency: sending a message from Roma to New York requires ~23 ms
  - We cannot completely hide failures in a large-scale DS
    - Cannot distinguish a slow server from a failing one
    - Cannot be sure that a server actually performed an operation before crashing
  - Price for achieving full transparency may be too high in term of performance
    - e.g., keeping data replicas *exactly* up-to-date takes time
    - e.g., *immediately flushing* write operations to disk for fault tolerance affects performance

  Trade-off between consistency and performance

Challenge 3: Openness

- Open DS: offers components that can easily be used by or integrated into other systems; consists of components that originate from elsewhere
- System should conform to well-defined interfaces
  - Defined through IDL (**Interface Definition Language**)
    - IDL is used to define data structures, interfaces (APIs), serialization formats in a language-agnostic way: complete and neutral
    - IDL examples: Protobuf, XDR, Thrift
- Benefits of openness
  - Interoperability
  - Portability of applications
  - Extensibility
- Examples: Jakarta EE, .Net, Web Services

Practice shows that many distributed systems are not as open as we'd like (van Steen & Tanenbaum)

Separation of policies and mechanisms

- To implement open and flexible DS, we need to organize the DS as a collection of relatively small and easily replaceable or adaptable component rather than as a monolithic system
- How? Separate policies from mechanisms
  - Policies refer to the principles guiding the decisions, while mechanisms represent the means to implement policies
- E.g., caching in a web browser:
  - Mechanism: store data and allow (dynamic) setting of caching policies
  - Caching policies:
    - Where to cache data?
    - How to free space when cache fills up?
    - When to refresh cached data?
    - Private or shared cache?

Separation of policies and mechanisms

- The other side of the coin
  - Strict separation can be counterproductive: the stricter the separation between policies and mechanisms, the more we need to ensure proper mechanisms, potentially leading to many configuration parameters and complex management
- Need to find a balance
- Possible solution: *self-configurable systems*

> *Finding the right balance in separating policies from mechanisms is one of the reasons why designing a distributed system is sometimes more an art than a science* (van Steen & Tanenbaum)

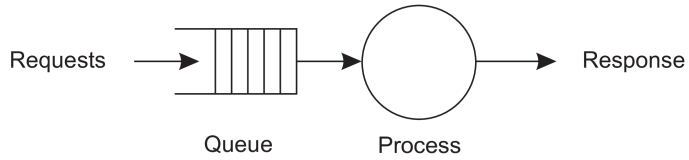
Challenge 4: Scalability

- **Scalability** is the property of a (distributed) *system* to keep an adequate level of performance notwithstanding a growing amount of:
  - Number of users and resources (**size** scalability)
  - Maximum distance between nodes (**geographical** scalability)
  - Number of administrative domains (**administrative** scalability)
- Most systems account only, to a certain extent, for size scalability

> *Many developers of modern distributed systems easily use the adjective scalable without making clear why their system actually scales.* (van Steen)

Size scalability

- Why a centralized system often lacks scalability
  - Computational capacity, limited by CPUs
  - Storage capacity, including transfer rate between CPUs and disks
  - Network between users and centralized system
- Formal analysis (see PMCS course)

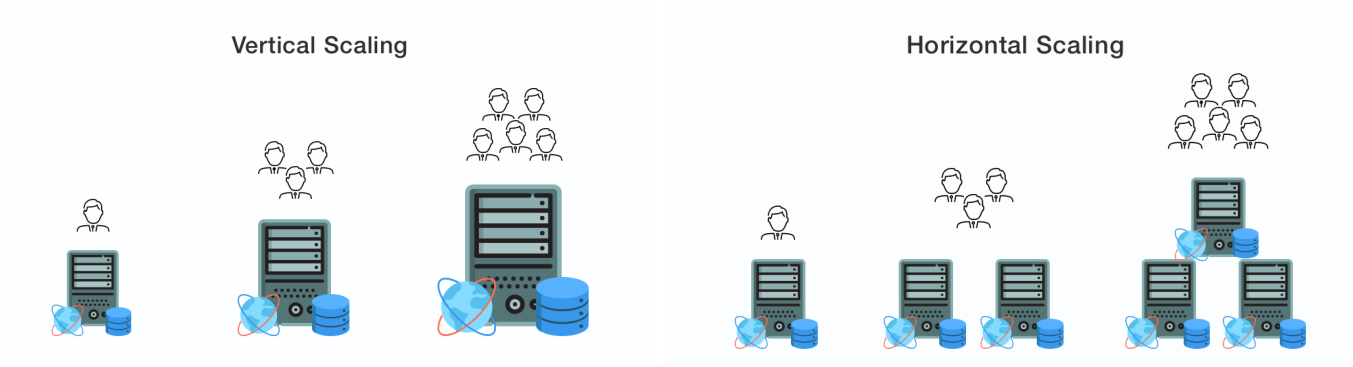


$$\frac{R}{S} = \frac{1}{1-U}$$

  - Service time S
  - Utilization U: fraction of time the service is busy
  - Response time R: total time take to process a request its arrival
  - If U is small, response-to-service time is close to 1: request is immediately processed
  - If U goes up to 1, system comes to a grinding halt. Solution: decrease S. How?

Size scalability

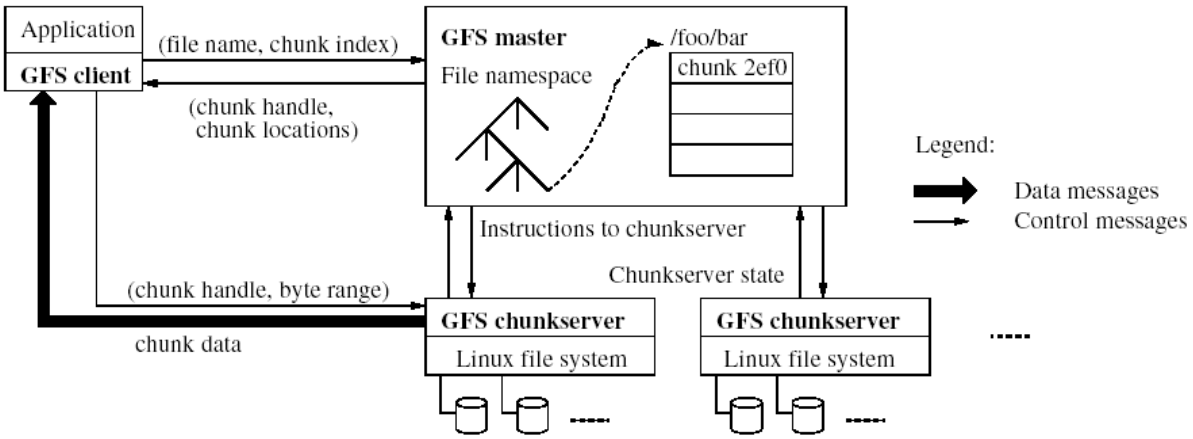
- Two directions for size scalability
  - **Vertical (scale-up)**: more powerful resources
  - **Horizontal (scale-out)**: more resources with same capacity



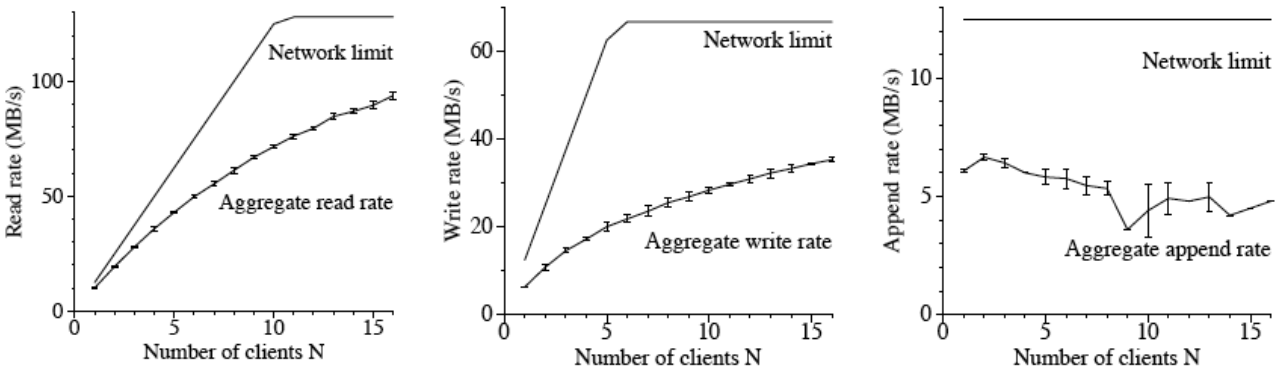
- Which resources? Physical, virtual (virtual machines, software containers, etc.)

Size scalability: GFS example

- Google File System (GFS)
  - Distributed file system based on divide et impera principle and horizontal scalability
  - Read, write, append operations
  - File divided into fixed blocks (chunks), which are distributed and replicated on multiple servers
  - Single master, multiple workers (chunkservers)



Size scalability: GFS example



(a) Reads (b) Writes (c) Record appends

- See experimental results
  - Scale parameter: number of clients
  - Scalability metric: aggregated read/write/append throughput (random file access)
  - Scalability criterion: the closer to network limit, the better
- Scalability is challenging!

Scaling techniques

1. Hide communication latency
   - Use asynchronous communication
   - Have separate handler for incoming response
   - Problem: not every application fits this model

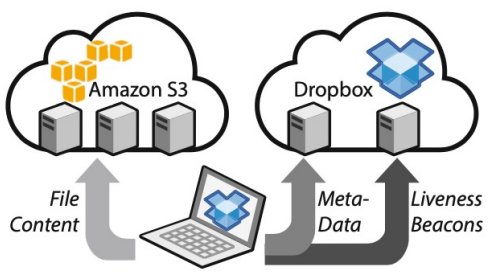
2. Move computation to clients

3. Distribute data and computation across multiple resources
   - Divide et impera: partition data and computation into smaller parts and distribute them across multiple DS resources
   - E.g., decentralized naming service (DNS), data-intensive distributed computation (Hadoop MapReduce and Spark)

Scaling techniques

4. Replicate DS resources and data
   - Make copies of data available at different DS nodes
   - Distribute processing on multiple DS nodes
   - Examples:
     - Distributed file systems and databases
     - Replicated Web servers
     - Web caches (in browsers and proxies)

- Example: in a cloud storage service (e.g., Dropbox, OneDrive, GDrive) data are locally cached on your device and replicated across multiple cloud servers



The problem with replication

- Applying replication is easy, but
- Having multiple copies leads to inconsistency: modifying one copy makes that copy different from the rest
- Trade-off: depending on application type, a certain degree of inconsistency can be tolerated
  - Blog, shared file, shopping cart, on-line auction, air traffic control
- We will study different consistency models to choose from

Challenge 5: Dependability

- **Dependability** refers to the degree that a computer system can be relied upon to operate as expected
  - partial failures make it intricate for distributed systems
- Requirements related to dependability
  - **Availability**: readiness for usage
  - **Reliability**: continuity of service delivery
  - **Safety**: very low probability of catastrophic consequences
  - **Maintainability**: how easily a failed system can be repaired

Dependability: availability

Can I use the system now?

- System is ready to use (i.e., operational) immediately
- Availability $A(t)$ of component C: probability that C is functioning correctly at time t

- Availability = uptime / (uptime + downtime)

- Normally expressed as number of 9's https://uptime.is/
  - A = 99%: two nines

    1-A=1% average days per year

    downtime per year = 0.01*365.2425 d = 3d 15h 39m 29.5s
  - A = 99.99%: four nines

    downtime per year = 0.0001*365.2425 d = 52 m 35.7 s

Dependability: availability

- Metrics

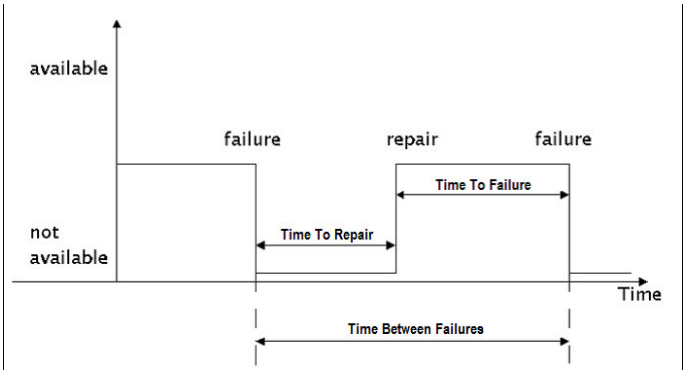
  A = MTTF/(MTTF + MTTR)

  Mean Time To Failure (MTTF): average time until a component fails

  Mean Time To Repair (MTTR): average time needed to repair a component

  Mean Time Between Failures (MTBF): MTTF + MTTR

  MTBF = total operating time / number of failures



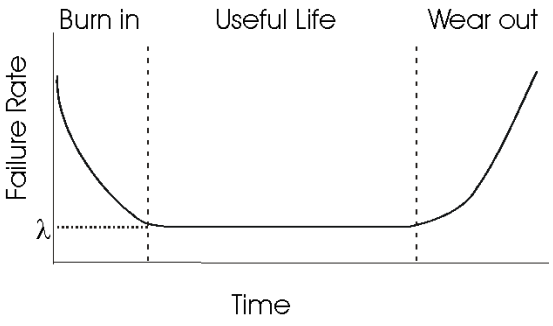
Dependability: reliability

Will the system be up as long as I need it?

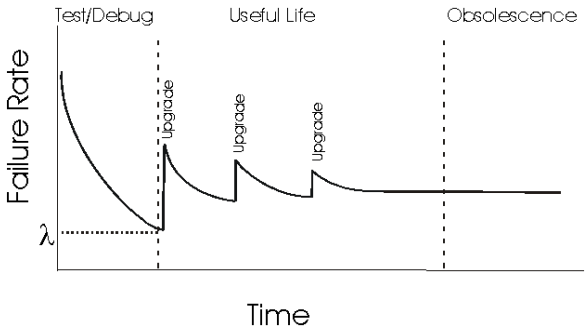
- System will run continuously without failure
- Reliability $R(t)$ of component C: conditional probability that C has been functioning correctly during $[0,t)$ given C was functioning correctly at the time $T = 0$

- Metrics: MTTF (and failure rate)

Bathtub curve for hardware



Revised bathtub curve for software



Availability vs reliability

- Availability ≠ reliability (when the system is repairable)

- Example A: system that goes down for 1 ms every hour
  - Highly available: > 99,9999% (= 1 - 1/(3600*1000))
  - Unreliable, because MTBF = 1 hour and there are 24*365=8780 failures per year

- Example B: system that never crashes but is shutdown for 2 weeks every year
  - Highly reliable, because MTBF = 1 year and there is only 1 failure per year
  - But only 96% available (= 1 - 14/365)

Failure, error and fault

- **Failure**: the system (or a component) is not living up to its specifications
  - Example: software crashes or produces wrong result
  - Can be cascading
- **Error**: wrong internal state, may lead to failure
  - Example: memory stores wrong value
- **Fault**: cause of an error
  - Example: memory is damaged
  - Can be transient, intermittent, permanent
- Relationship: fault $\rightarrow$ error $\rightarrow$ failure

Dependability: tools

- **Fault prevention**
  - Prevent fault occurrence
- **Fault tolerance**
  - Build component and make it mask fault occurrence
- **Fault removal**
  - Reduce presence, number, or seriousness of faults
- **Fault forecasting**
  - Estimate current presence, future incidence, and consequences of faults

Challenge 6: Security

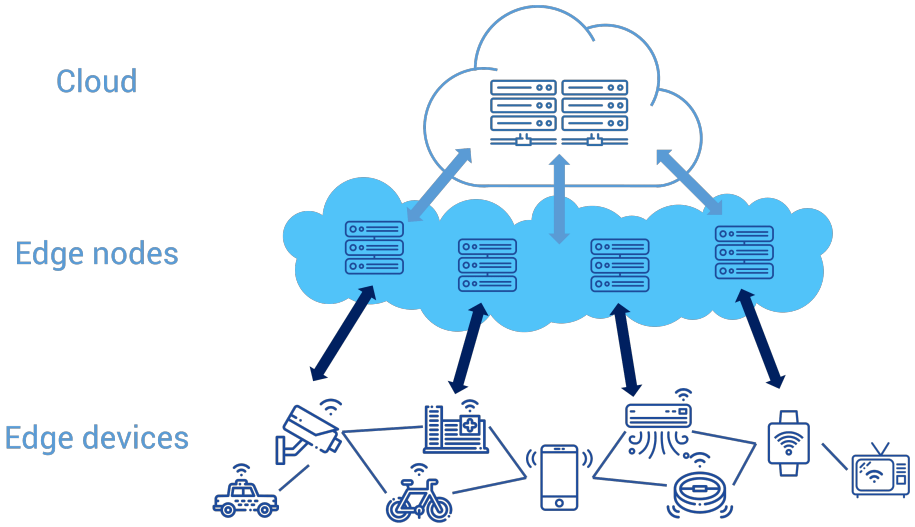
- A distributed system that is not secure, is not dependable
- What we need
  - **Confidentiality**: information is disclosed only to authorized parties
  - **Integrity**: ensure that alterations to system assets can be made only in an authorized way
- Authorization, authentication, trust
  - Authentication: verifying correctness of claimed identity
  - Authorization: does an identified entity has proper access rights?
  - Trust: one entity can be assured that another will perform particular actions according to a specific expectation

Security mechanisms

- Keeping it simple: it is all about encrypting and decrypting data using security keys
- Symmetric vs asymmetric cryptosystem
  - Symmetric: same encryption and decryption key, both keys should be kept secret
  - Asymmetric: public key and private key
- Secure hashing
  - In practice, we use secure hash functions: $H(data)$ returns a fixed-length string
    - Any change from *data* to *data*$*$ will lead to a completely different string $H(data*)$
    - Given a hash value h, it is computationally infeasible to find a *data* with $h = H(data)$

Edge computing

- Brings computation and storage at the network edges, in proximity of data producers (e.g., IoT devices) and consumers (e.g., users)



Distributed pervasive systems

- Distributed systems where computing is embedded across many devices, which are
  - often small, mobile, embedded, battery-powered
  - often spread out geographically
  - coordinated to work together seamlessly
    - The system naturally blends into the user's environment
- Sub-categories:
  - Ubiquitous computing systems: pervasive and continuously present (continuous interaction with users), e.g., smart home system
  - Mobile computing systems: pervasive with mobile devices, e.g., fleet of drones for emergency management
  - Sensor networks: pervasive, that work together to sense and monitor data about environmental or physical conditions, e.g., air quality

Pitfalls in realizing distributed systems

- Many distributed systems are needlessly complex because of errors in design and implementation that were patched later
- Common wrong assumptions by architects and designers of distributed systems ("The Eight Fallacies of Distributed Computing", Peter Deutsch, 1991-92):
  1. The network is reliable
     - "You have to design distributed systems with the expectation of failure" (Ken Arnold)
  2. Latency is zero
     - Latency is more problematic than bandwidth
     - "At roughly 300,000 km/s, it will always take at least 30 ms to send a ping from Europe to the US and back, even if the processing would be done in real time." (Ingo Rammer)
  3. Bandwidth is infinite

Pitfalls in realizing distributed systems

4. The network is secure
5. The topology does not change
   - That's right, it doesn't - as long as it stays in the test lab!
6. There is one administrator
7. Transport cost is zero
   - Going from the application level to the transport level is not free
   - Costs for setting and running the network are not free
8. The network is homogeneous

Technology is not the solution to everything!

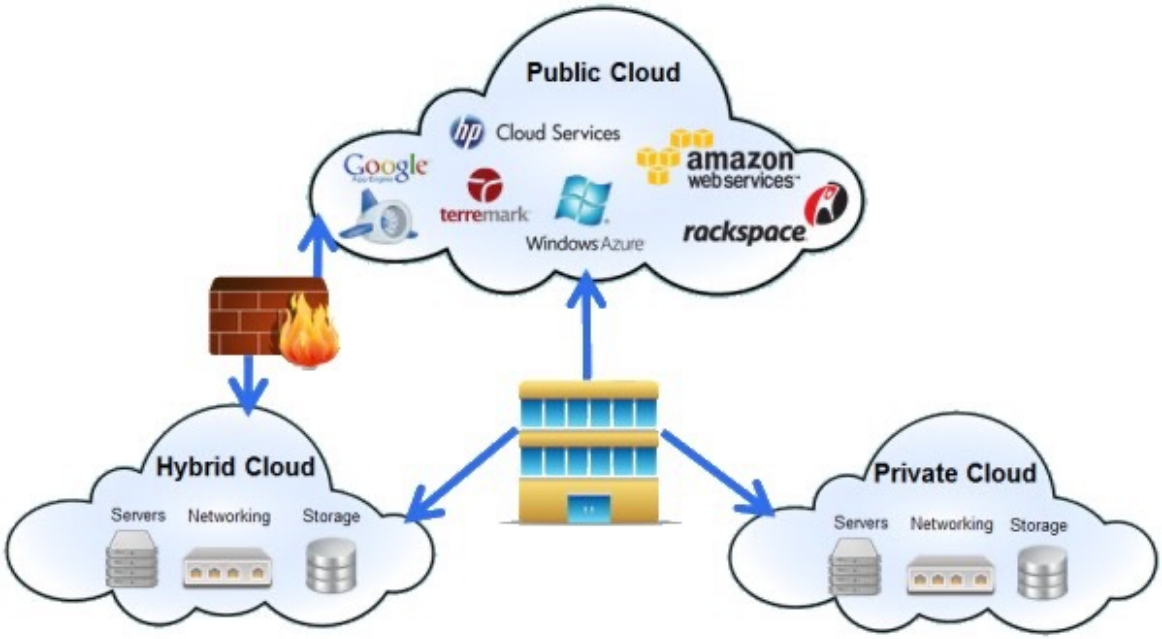
… sharing some essential characteristics

- **On-demand self-service**
  - Cloud resources can be provisioned on-demand by users, without requiring interactions with cloud provider
- Broad **network access**
  - Cloud resources accessed over Internet using standard access mechanisms that provide platform-independent access
  - Published service interface/API
- **Elasticity**
  - Ability for customers to quickly request, receive, and later release as many resources as needed
  - Cloud resources can be provisioned rapidly and elastically: they can be rapidly scaled out/in based on demand

… that share some essential characteristics

- Resource **pooling**
  - Cloud resources are pooled to serve multiple users using multi-tenancy
  - Multi-tenancy: multiple users served by the same physical hardware
- Resource **virtualization**
  - Resources: storage, processing, memory, network bandwidth, and even data centers
- **Pay-per-use** pricing model
  - No large up-front acquisition cost
- **Measured** service
  - Usage of cloud resources is measured and user is charged based on some specific metric

Cloud deployment models



Deployment models: public

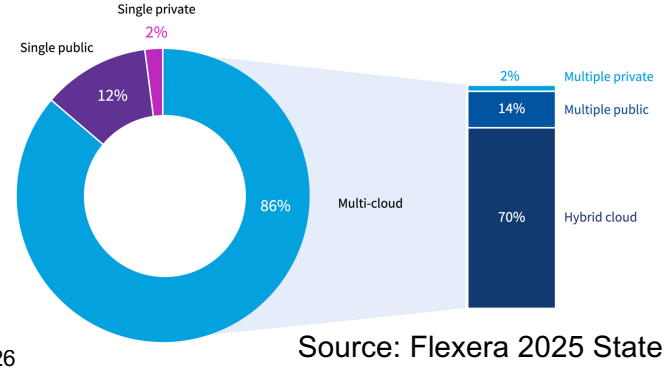
- Public cloud
  - Cloud infrastructure: provisioned for open use by the general public (*multi-tenant*)
  - Owned, managed, and operated by a business, academic, or government organization, or some combination of them
  - Exists on premises of cloud provider
  - Services can be free or fee-based

Deployment models: private

- Private/enterprise cloud
  - Cloud infrastructure: provisioned for exclusive use by a single organization (*single-tenant*) comprising multiple consumers (e.g., business units)
  - Owned, managed, and operated by the organization, a third party, or some combination of them
  - Exists on- or off-premises of cloud provider
  - ✓ Stronger security, customization
  - ✗ Lower economic advantages, scalability more laborious

Deployment models: hybrid

- Hybrid cloud
  - Mixed use of private and public clouds
  - Cloud infrastructure: composition of two or more distinct cloud infrastructures (private or public) that
    - remain unique entities
    - but are bound together by standardized or proprietary technology that enables data and application portability
  - The most popular deployment model



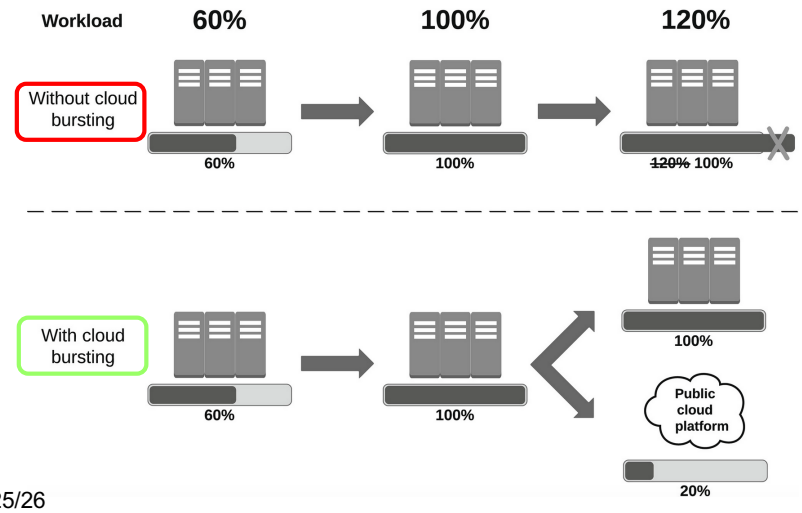
Source: Flexera 2025 State of Cloud Report

Deployment models: hybrid

- Pros
  - ✓ Balance resources and costs
  - ✓ Increase flexibility
    - Differentiate privacy, e.g., sensitive data is kept in private cloud
    - Differentiate services: apps are siloed on different clouds
    - Improve availability: public cloud for disaster recovery in case of unexpected outage
    - Improve geographic distribution and compliance: public cloud resources in different geographic regions
    - Improve performance: by means of cloud bursting
  - ✓ Reduce vendor lock-in risks
    - Vendor lock-in happens when someone is forced to continue using a service regardless of quality, because switching away from that service is not practical

Hybrid cloud: cloud bursting

- Use hybrid cloud dynamically to manage workload spikes when private cloud capacity is insufficient
  - Workload runs in a private cloud which reaches its peak capacity
  - Use public cloud to handle workload spikes without maintaining excess capacity on-premise



Service models: IaaS

- Infrastructure as a Service (IaaS)
  - Compute, storage and network resources as services
  - Customer capability
    - To provision fundamental computing resources (processing, storage, networks) where consumers deploy and run arbitrary software (including OS and apps)
  - Customer control or management
    - No control over underlying cloud infrastructure
    - Control over OS, storage, deployed apps
    - Limited control of selected network components (e.g., firewalls)

IaaS: examples

- Amazon Web Services (AWS): e.g., EC2, S3
- Google Cloud: e.g., Compute Engine, Cloud Storage
- Alibaba Cloud: e.g., Elastic Compute Service
- Aruba Cloud: e.g., Virtual Data Center
- CloudSigma
- DigitalOcean
- IBMCloud : e.g., Cloud Virtual Servers, Cloud Object Storage
- Microsoft Azure: e.g., Virtual Machines, Storage

IaaS: AWS EC2

- Let us launch a virtual machine on EC2 from AWS Management Console
- Few steps in 2-3 minutes:
  1. Sign in to AWS Management Console https://signin.aws.amazon.com/
  2. Open Amazon EC2 console by choosing EC2 under Compute
  3. Choose AWS Region for instance deployment
  4. Choose Launch Instance
  5. Choose software configuration (operating system and other sw) from available Amazon Machine Images (AMIs)
     - AMI provided by AWS, user community, or AWS Marketplace
  6. Choose resource configuration
     - T-shirt sizes: various combinations of CPU, GPU, memory, storage, and networking capacity
  7. If not yet configured, select security group to open a specific network port (e.g., SSH) to access the instance
  8. Select an existing key pair or create a new one to securely connect to the instance after it launches

Service models: PaaS

- Platform as a Service (PaaS)
  - Allows customers to develop, build, run and scale applications, without the complexity of building and maintaining the underlying infrastructure
  - Customer capability
    - To develop, deploy and test onto the cloud (consumer-created or consumer-acquired) apps realized using programming languages, application frameworks, development tools supported by PaaS provider
  - Customer control or management
    - No control over underlying cloud infrastructure (network, servers, OS, storage)
    - Control over deployed apps and possibly application hosting environment configurations

PaaS: examples

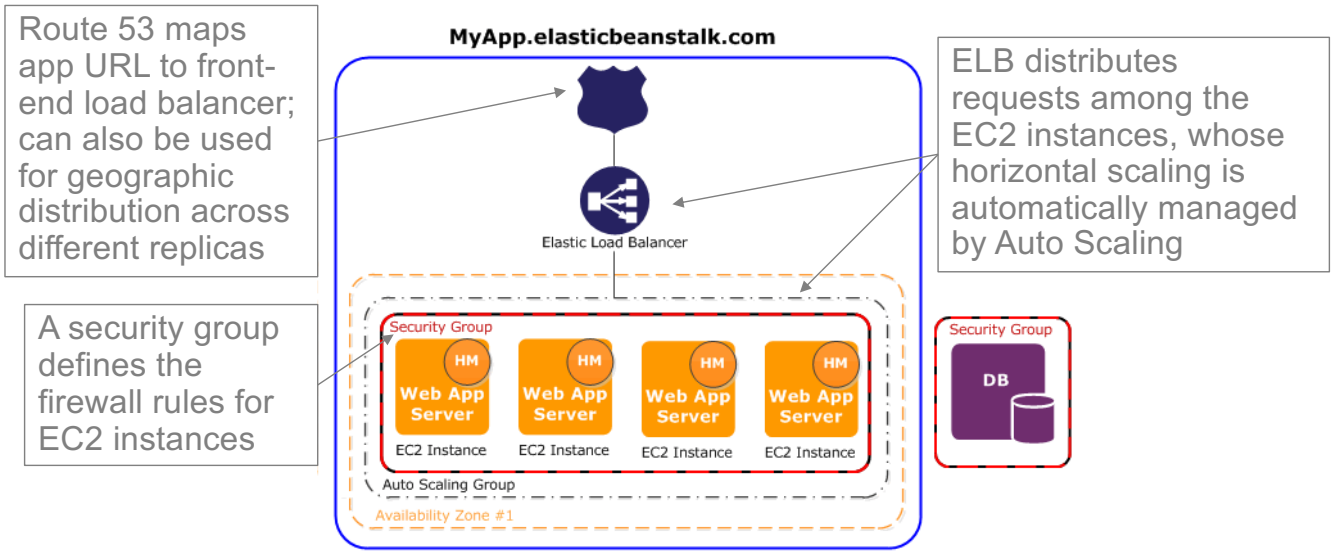
- Platforms for Web apps
  - AWS Elastic Beanstalk, Google App Engine, Azure App Service
- Platforms for building and deploying applications in containers and Kubernetes
  - AWS Elastic Container Service, Google Kubernetes Engine, Azure Kubernetes Service
- Platforms for entire application lifecycle
  - Include tools for CI/CD, container orchestration, and integration with various services
  - Heroku, Red Hat OpenShift, VMware Tanzu
- Salesforce Platform
  - Allows developers to build, deploy, and manage custom applications within Salesforce ecosystem
- Serverless computing/Function as a Service (FaaS)
  - AWS Lambda, Google Cloud Functions, Alibaba Function Compute

PaaS: AWS Elastic Beanstalk

- Supports applications developed in Go, Java, .NET, Node.js, PHP, Python, and Ruby
- Allows user to deploy and manage applications without having to learn about the infrastructure
- Reduces management complexity
  - User uploads the application source bundle (e.g., .war file) to Elastic Beanstalk and provides some information about the application
  - Elastic Beanstalk automatically launches an environment and creates and configures the AWS resources needed to run the code, handling the details of capacity provisioning, load balancing, scaling, and application health monitoring

PaaS: AWS Elastic Beanstalk

- Example of Elastic Beanstalk architecture
  - AWS resources (ELB, Auto Scaling group, EC2 instances) are automatically provisioned by Elastic Beanstalk



https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/concepts-webserver.html

IaaS vs. PaaS

- Using IaaS
  - "Pure" virtualized resources: CPU, memory, storage, load balancer, ...
  - OS included
  - "T-shirt" size

- Using PaaS
  - Virtualized resources plus application framework
  - Additional services, e.g., horizontal scaling without having to configure it
  - ✗ Constraints on application and data architecture
  - ✗ Increased risk of vendor lock-in

IaaS and PaaS providers

- Main providers: AWS, Microsoft Azure, Google Cloud, IBM Cloud, Oracle, Alibaba
- Gartner's magic quadrant for strategic cloud platform services (2024)

Figure 1: Magic Quadrant for Strategic Cloud Platform Services



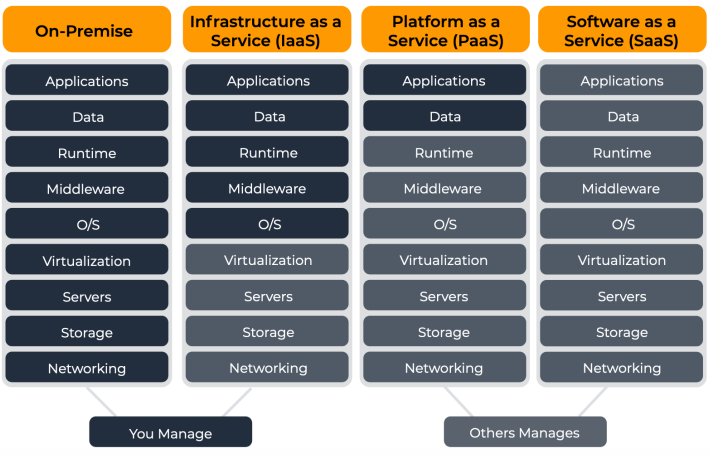
Service models: SaaS

- Software as a Service (SaaS)
  - Applications made available to customers over Internet, still the largest market
  - Customer capability
    - To use SaaS provider's applications running on a cloud infrastructure
    - Applications accessible from various client devices through Web or provider APIs
  - Customer control or management
    - No control over underlying cloud infrastructure
      - Network, servers, operating systems, storage, or even individual application capabilities
    - Possible exception: limited user-specific application configuration settings

SaaS: examples

- Communication & collaboration: Adobe Connect, Cisco WebEx, Google Mail, Zoom
- Personal productivity: Google Calendar, Google Drive, Microsoft 365
- File storage and sharing: Dropbox, OneDrive, iCloud, SugarSync
- CRM (Customer Relationship Management): salesforce.com
- Enterprise-level (accounting, ERP, HR, marketing, sales): NetSuite, SAP Business ByDesign, Workday, Zoho

Service models: wrapping up



- Additional service models, e.g.:
  - Database as a Service (DBaaS)
  - Function as a Service (FaaS)
  - Monitoring as a Service (MaaS)
  - … Anything as a Service (XaaS)

Cloud pricing models

- Cloud providers offer a number of pricing models

- Pay-per-use / on-demand
  - Users are charged based on cloud resources usage
  - Pricing granularity depends on service, e.g., by hour for AWS EC2, by ms for AWS Lambda
  - ✓ Maximum flexibility: no need to plan in advance

- Fixed / reserved
  - Users are charged a fixed amount per month for cloud resources
  - ✓ Significant discount compared to on-demand pricing
  - ✗ Upfront payment

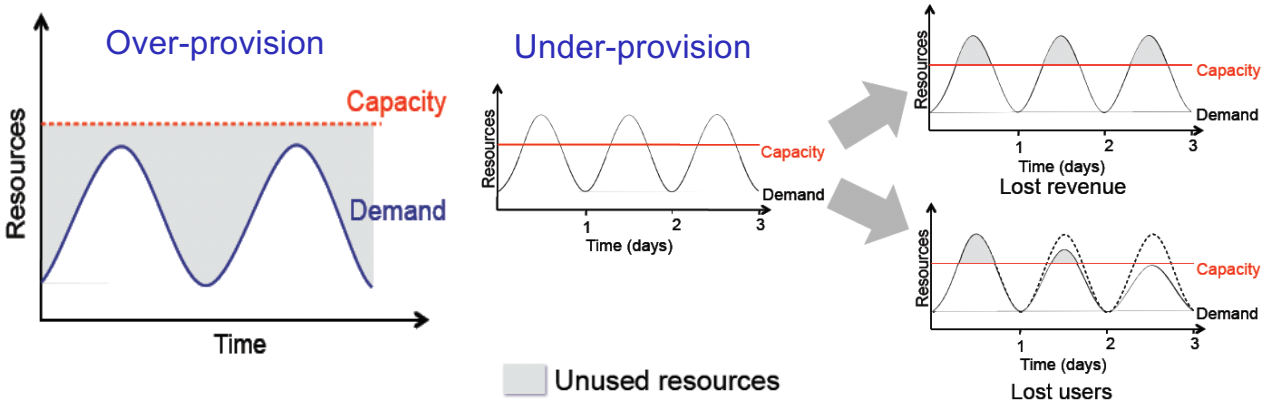
Cloud pricing models

- Dedicated
  - Physical resources (e.g., physical server) rather than virtual ones fully dedicated to user
  - ✓ Dedicated hardware, maximum isolation
  - ✗ Higher cost

- Spot
  - Variable pricing for cloud resources driven by market demand (supply and demand)
  - ✓ Substantial savings
    - E.g., AWS EC2 spot instances can be acquired at up a 90% discount compared to on-demand pricing https://aws.amazon.com/ec2/spot/
  - ✗ Interruptible by cloud provider
    - 👉 use for fault-tolerant workloads

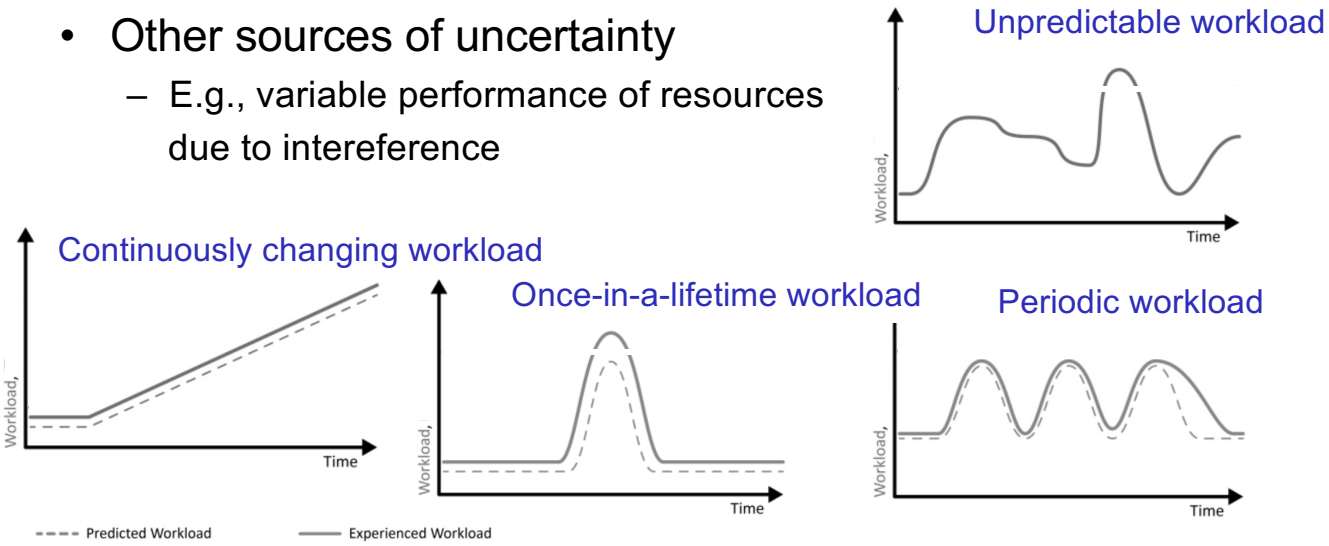
Capacity planning and resource provisioning

- Capacity planning: determines right sizing (of each tier) of application deployment in terms of:
  - Number of IT resources and their size
  - IT resources: computing, storage, memory and network
- Static provisioning: capacity is pre-provisioned
  - ✗ Excess capacity (over-provision based on peak load) or capacity lack (under-provision)



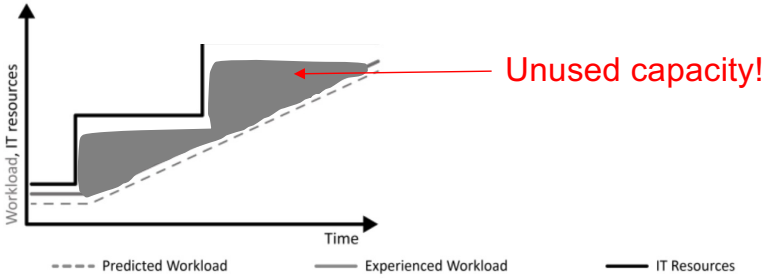
To make things more complicated: uncertainty

- Fluctuating workloads with different patterns
- Other sources of uncertainty
  - E.g., variable performance of resources due to intereference



A solution…

- Forecast demand
- Next, manually provision resources dynamically by scaling in/out or scaling up/down IT resources as needed

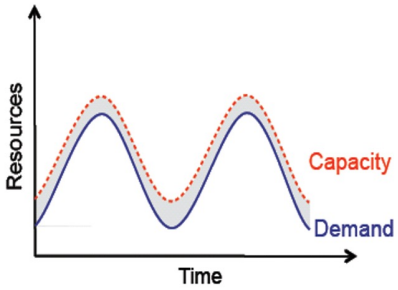


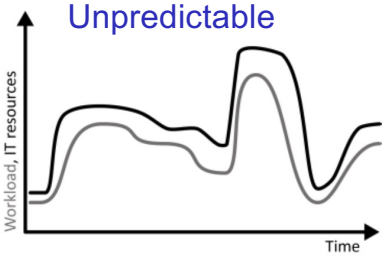
- Manual scaling: unpractical in cloud environments
  - Takes time, prone to human errors, inefficient, …

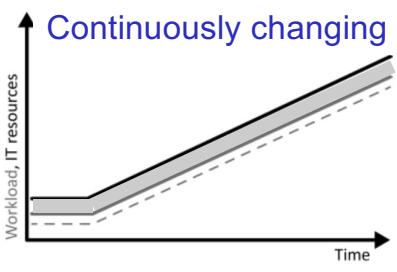
👉 Automated scaling is better!

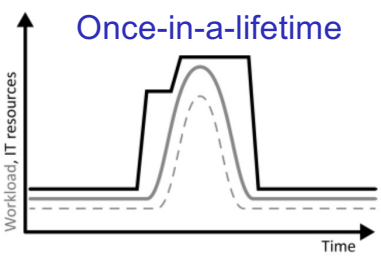
The solution: elastic scaling (aka auto-scaling)

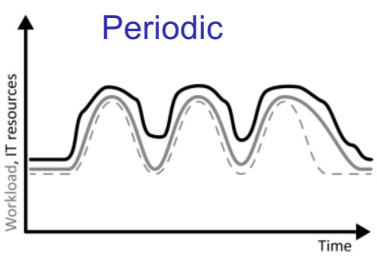
- Let's automatically align IT resources to changing workloads





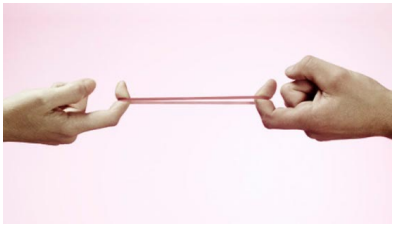






Elasticity

- (Physics) Property of returning to an initial form or state following deformation
  - Stretch when a force stresses it
  - Shrink when the stress is removed

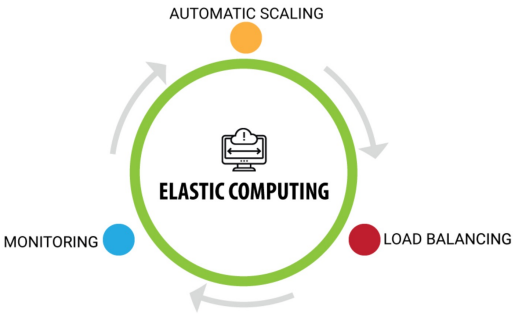


- (Computer science) Degree to which a computing system is able to adapt to workload changes by provisioning and de-provisioning resources in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible

Herbst et al., Elasticity in Cloud Computing: What It Is, and What It Is Not, ICAC 2013

Elasticity and Cloud computing

- Elasticity: a key feature of Cloud computing
- How do we achieve elastic computing?
- We need a self-adaptive software system

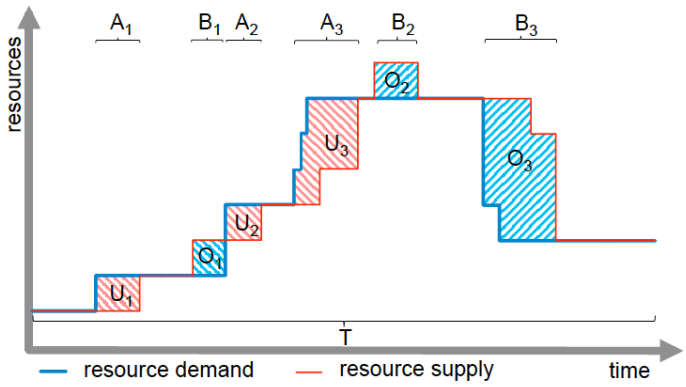


*In any system of sufficiently large scale, **automation** is not only necessary, but it is cheap.*

Ousterhout, Is scale your enemy, or is scale your friend?: technical perspective, Comm. ACM, 2011
https://dl.acm.org/doi/pdf/10.1145/1965724.1965748

Elasticity: how to measure

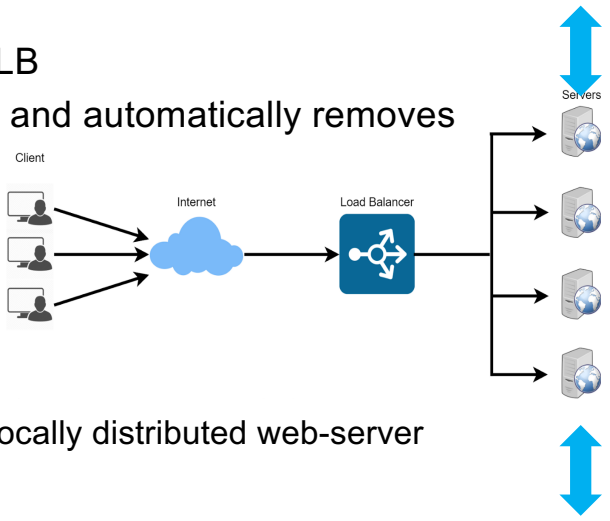
- Consider over-provision and under-provision



- Elasticity metrics:
  - Accuracy: sum of areas of over-provision (*O*) and under-provision (*U*) over the measurement period *T*
  - Timing: total amount of time spent in over-provision (*B*) and under-provision (*A*) over the measurement period *T*

Elasticity: architectural point of view

- Distribute requests among multiple server replicas using a *server-side* **load balancer** (LB)
  - A single external IP address (VIP) assigned to LB
  - Network (layer 4) or application (layer 7) LB
  - One-way LB (responses from replicas go directly to clients) or two-way LB (responses go through LB)
  - Hardware or software LB
  - Centralized or distributed LB
  - Health checks: LB detects and automatically removes unhealthy replicas



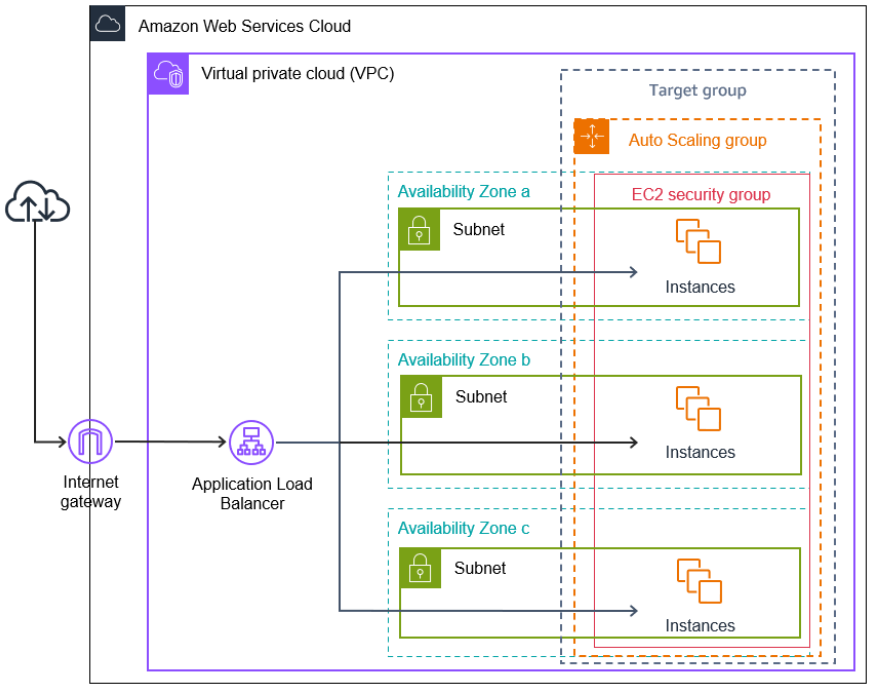
Cardellini et al., The state of the art in locally distributed web-server systems, ACM Comput. Surv., 2002

Elasticity: architectural point of view

- **Load balancing goals**
  - Maximize resource utilization
  - Minimize response time
  - Maximize throughput
- **Load balancing policies**
  - Stateless: Random, Round Robin
  - Server-aware: Least Loaded, Weighted Round Robin (assign weights proportional to server capacity and distribute requests in a circular way according to weights), Power of Two Choices (select randomly two servers and choose the least loaded of the two), …
  - Client-aware (depends on layer): Hashing, Session Stickiness, …
  - Client & server-aware: combination of client and server info
- Some products: Envoy https://www.envoyproxy.io, nginx https://nginx.org/en/docs/http/load_balancing.html

Elasticity: architectural point of view

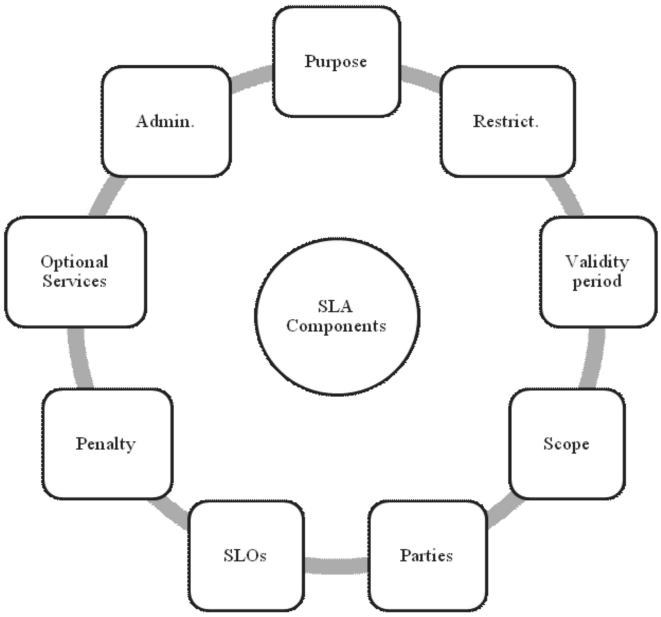
- Example of load balancing and auto-scaling using AWS



https://docs.aws.amazon.com/autoscaling/ec2/userguide/tutorial-ec2-auto-scaling-load-balancer.html

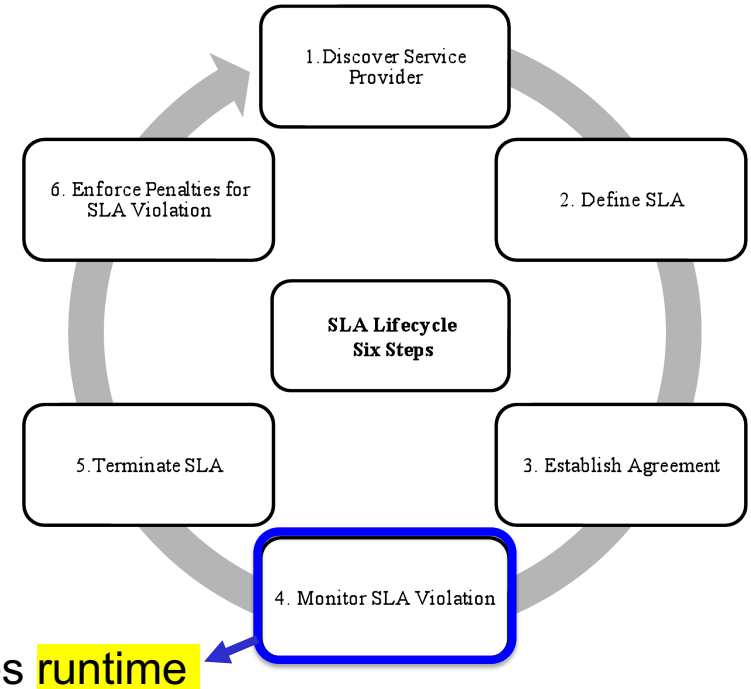
Service Level Agreement (SLA)

- Formal agreement (contract) between a provider and a consumer of a service
- Composed of one or more Service Level Objectives (SLOs):
  - SLO: condition on a measure of a specific metric (e.g., mean response time <= 1 s)
- Includes penalty and/or compensation in case of SLA violation



Source: Sun Microsystems Internet Data Center Group

SLA life cycle



Requires runtime monitoring of service

SLI and SLO

- Service Level Indicator (SLI)
  - A measureable metric
  - Examples: availability, response time, throughput
  - Availability is the most common SLI for cloud services

- Service Level Objective (SLO)
  - A predicate over a set of SLIs
  - Example: a fixed threshold on a single SLI
  - Example SLO:

monthly uptime percentage for a VM is at least 99.99%

SLI — SLO

Example: Amazon Compute SLA

- **Service Commitment** https://aws.amazon.com/compute/sla/?nc1=h_ls

  For each individual Amazon EC2 instance (“Single EC2 Instance”), AWS will use commercially reasonable efforts to make the Single EC2 Instance available with an Instance-Level Uptime Percentage of at least 99.5%, in each case during any monthly billing cycle (the “Instance-Level SLA”). In the event any Single EC2 Instance does not meet the Instance-Level SLA, you will be eligible to receive a Service Credit

Instance-Level Uptime Percentage	Service Credit Percentage
Less than 99.5% but equal to or greater than 99.0%	10%
Less than 99.0% but equal to or greater than 95.0%	30%
Less than 95.0%	100%

  Monthly Uptime Percentage 99.5% = 3h 39m 8s of downtime per month

  Monthly Uptime Percentage 99% = 7h 18m 17s of downtime per month

- **Service Credits**

  Service Credits are calculated as a percentage of the monthly bill… We will apply any Service Credits only against future payments… To receive a Service Credit, you must submit a claim…

Example: Amazon S3 SLA

https://aws.amazon.com/s3/sla/?nc1=h_ls

- **Service Commitment**

  AWS will use commercially reasonable efforts to make Amazon S3 available with a Monthly Uptime Percentage during any monthly billing cycle. In the event Amazon S3 does not meet the Service Commitment, you will be eligible to receive a Service Credit as described below.

  Monthly Uptime Percentage 99.9% = 43 min 49 sec of downtime per month

- **Definitions**
  - "Error Rate" means, for each request type to an Amazon S3 storage class: (i) the total number of internal server errors returned by Amazon S3 for such request type to the applicable Amazon S3 storage class as error status "InternalError" or "ServiceUnavailable" divided by (ii) the total number of requests for such request type during the applicable 5-minute interval of the monthly billing cycle. We will calculate the Error Rate for each AWS account as a percentage for each 5-minute interval in the monthly billing cycle. The calculation of the number of internal server errors will not include errors that arise directly or indirectly as a result of any of the Amazon S3 SLA Exclusions.
  - "Monthly Uptime Percentage" is calculated by subtracting from 100% the average of the Error Rates from each 5-minute period in the monthly billing cycle. If you did not make any requests in a given 5-minute interval, that interval is assumed to have a 0% Error Rate.

Issues with cloud SLAs

- Another example: https://cloud.google.com/compute/sla
- Issues related to cloud SLAs:
  - SLA jargon: confusing terminology
  - Availability
    - Averaging over a period can obscure the difference between many short outages vs. a few long ones
  - Lack of SLIs for response time
  - Service credits: only applicable to future payments
  - SLA violation detection: burden of detecting violations often on cloud customer
  - Time period for reporting SLA violations
  - Non-negotiable (or customizable) SLA for most users
  - Difficulty in comparing SLAs across different cloud providers

Cloud monitoring

- Goal: continuously track and assess health of systems and services deployed within cloud
  - Ensure that any performance issues, outages, or inefficiencies are quickly detected and addressed to maintain optimal system performance and availability
- Requires tools to collect and analyze data on system-oriented and application-oriented metrics from cloud resources and services
- System-oriented metrics

Type	Metrics
CPU	CPU utilization
Disk	Disk utilization, throughput (MB/sec for read/write, operations/sec)
Memory	Memory-Used, Memory-Free, Page-Cache
Interface	Throughput (Mbps incoming/outgoing)

- Application-oriented metrics: response times, error rates, transaction volumes, user interactions, …

Cloud monitoring: services and tools

- Examples of monitoring services offered by cloud providers
  - Amazon CloudWatch https://aws.amazon.com/cloudwatch
    - Provides monitoring for AWS cloud resources and applications, including system-level and applications metrics
    - Supports alarms, automated responses, and detailed logging for troubleshooting
  - Google Cloud Operations Suite https://cloud.google.com/monitoring
- Examples of third-party tools
  - Prometheus https://prometheus.io
    - Open-source monitoring tool, collects and stores metrics in a time series DB, which can then be queried for real-time analysis
  - Datadog https://www.datadoghq.com
  - Dynatrace https://www.dynatrace.com

Outside a cloud data center





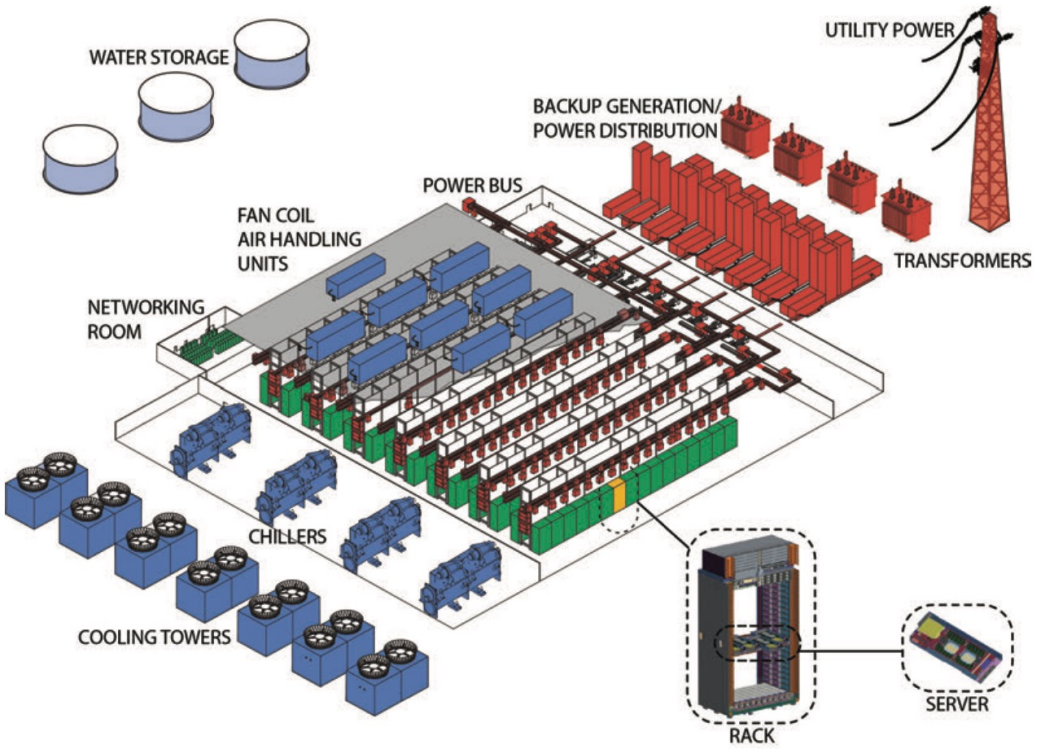
Cooling towers

- How large? ~100000 m^2
- How many servers? ~10^6
- Power-efficient?

Power Usage Effectiveness (**PUE**) = Facility power / IT equipment power

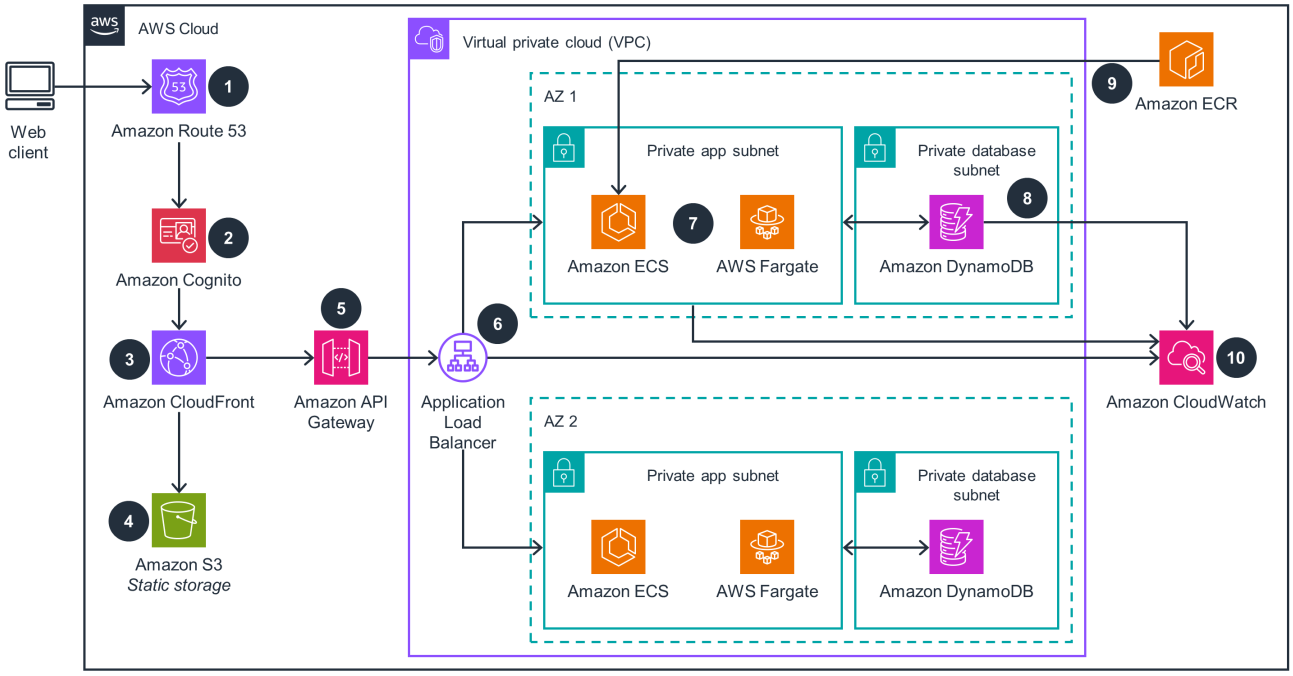
  - E.g., average PUE for Google data centers is 1.1

Main components of a cloud data center



Example of web application in cloud

- A scalable, secure 3-tier web application using containerization on AWS



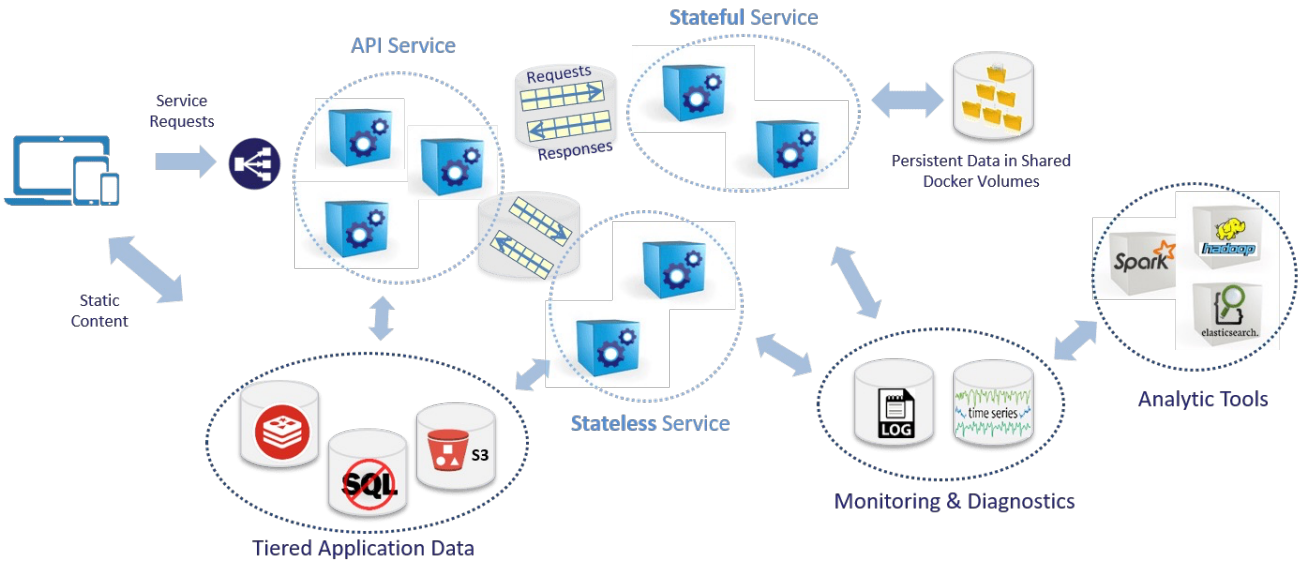
https://aws.amazon.com/it/solutions/guidance/building-a-containerized-and-scalable-web-application-on-aws

Cloud-native applications

- Cloud native: sw approach of building, deploying, and managing modern applications in cloud environments
- Cloud-native applications are a collection of small, independent, and loosely coupled services
- Goals: agile, highly scalable, flexible, and resilient applications
- We will study microservices and serverless applications

Cloud-native applications: high-level example

Stateful service: retain data or context between sessions or interactions



Cloud trends

- AI and ML Cloud services, Container-as-a-Service, serverless are rising
- Public cloud adoption still growing
- Multi-cloud is popular: concurrent usage of multiple clouds
  - Hybrid, multiple private, multiple public

Multi-cloud: challenges

- Multi-cloud management is more complex
  - Managing resources spread across different clouds
  - Automating workload placement, configuration and maintenance (including identity management and data protection/encryption)
  - Monitoring performance, resource usage and costs of multiple infrastructures and applications
  - Assessing changes in service portfolio and pricing of cloud providers (FinOps)
  - Predicting performance, resource usage and costs
- How to deploy multi-cloud applications?
  - Commercial purpose-built products
  - **Infrastructure as Code (IaC)**: tools like Terraform, Ansible, Chef, or Puppet allow you to define cloud infrastructure in code and deploy it consistently across multiple cloud providers

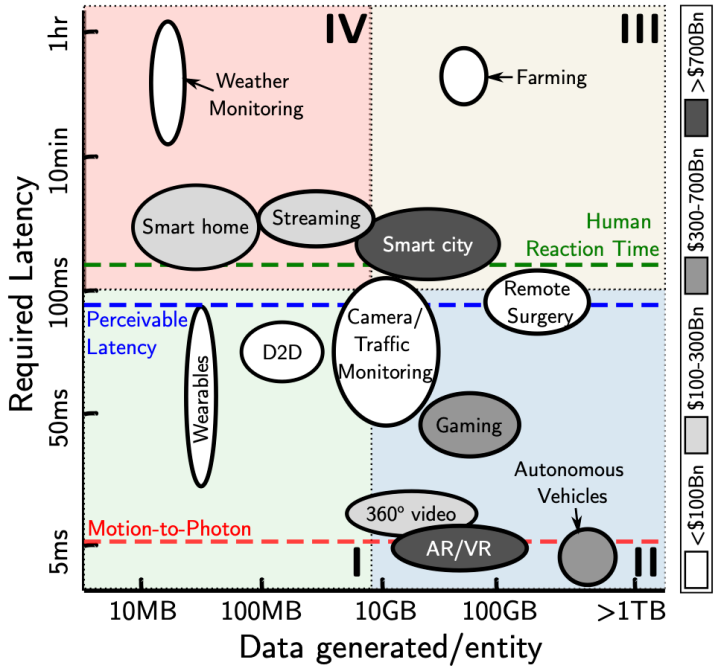


Cloud trends: applications

- Convergence of different technologies, evolving landscape
- Current applications
  - IoT, Big Data, AI and ML
- Emerging applications
  - Generative AI
  - Metaverse: collective virtual 3D shared space, created by the convergence of virtually enhanced physical and digital reality
  - Web3: stack of technologies built on blockchain protocols that support the development of decentralized web applications and empower users to control their own identity, content, and data
  - Quantum computing: tasks like cryptography, optimization, and simulation of complex molecules

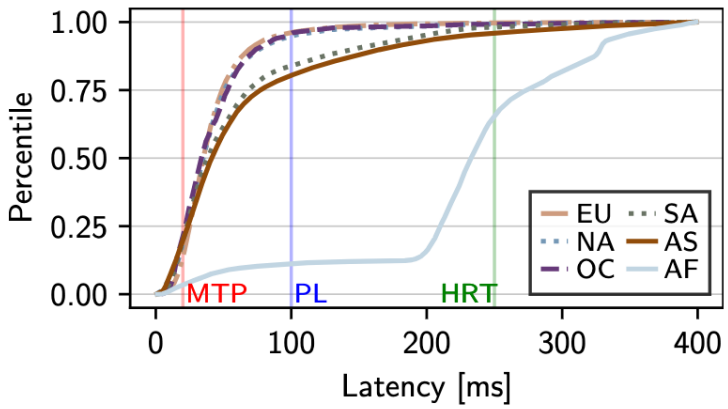
Does cloud suit every application?

- Different classes of applications, different performance requirements



Beyond cloud

- Challenge with latency-sensitive and location-aware applications
  - Apps like real-time manufacturing, self-driving vehicles, drones, cognitive assistants generate massive data volumes
  - Relying on cloud can be impractical due to high network latency
  - See round-trip times (RTTs) to nearest cloud data center



Motion-to-Photon (MTP)
Human Perceivable Latency (PL)
Human Reaction Time (HRT)

Source: Cloudy with a chance of short RTTs: analyzing cloud connectivity in the internet, https://tinyurl.com/3wztmmz6

Computing continuum: challenges

- Infrastructure and application management: interoperability and seamless integration
- Resource heterogeneity (capacity, energy, ...)
- Performance guarantee
- Security and privacy

The Computing Continuum

IoT/Edge — Fog — HPC/Cloud/Instrument

Size	Nano	Micro	Milli	Server	Fog	Campus	Facility
Example	Adafruit Trinket	Particle.io Boron	Array of Things	Linux Box	Co-located Blades	1000-node cluster	Datacenter
Memory	0.5K	256K	8GB	32GB	256G	32TB	16PB
Network	BLE	WiFi/LTE	WiFi/LTE	1 GigE	10GigE	40GigE	N*100GigE
Cost	$5	$30	$600	$3K	$50K	$2M	$1000M

Count = 10^9 Size = 10^1 … Count = 10^1 Size = 10^9

Beckman et al., Harnessing the Computing Continuum for Programming Our World, 2020 https://www.netlib.org/utk/people/JackDongarra/PAPERS/icl-utk-1358-2020.pdf

- Methodologies, programming models, tools and platforms for computing continuum are still in their infancy

Summing up: cloud benefits

IT benefits

- Scalability, elasticity, flexibility
- Anytime and anywhere accessibility
- Improved security
  - E.g., 3.47 Tbps DDoS attack from 10,000 servers successfully stopped in 2022 https://tinyurl.com/bdec45dy
- Organizational and operation agility
- Simplified management
  - E.g., software updates and versioning

Business benefits

- No advanced payment
  - Costs scale with use
  - CapEx to OpEx
- Rapid business innovation
- Increase in productivity

Issues for cloud customers

- Privacy, security and legal issues
  - Control over data: location of data centers, data access
    - Data sovereignty: data are subject to the laws and governance structures of the nation where they are collected
  - Data encryption, also in transit
  - Data integrity, tracing and recovery
  - GDPR compliance (accountability, privacy by design, privacy by default)

- We had “stormy” clouds
  - “You have zero privacy anyway. Get over it.” (Scott McNealy, co-founder of SUN, 1999)
  - “If you have something that you don’t want anyone to know, maybe you shouldn’t be doing it in the first place.... The reality is that search engines do retain information... It could become available later...” (Eric Schmidt, Google CEO, 2009)

Issues for cloud customers

- Now?
  - In Europe: CISPE Code of Conduct
    https://www.codeofconduct.cloud/
  - Be aware of privacy policies
    - "We use the information we collect in existing services to help us develop new ones. For example, understanding how people organized their photos in Picasa, Google’s first photos app, helped us design and launch Google Photos." (Google, 2024)

      https://policies.google.com/privacy
    - "Microsoft uses the data we collect to provide you with rich, interactive experiences. (…) We also use the data to operate our business, which includes analyzing our performance, meeting our legal obligations, developing our workforce, and doing research." (Microsoft, 2024)

      https://privacy.microsoft.com/en-gb/privacystatement

Issues for cloud customers

- Network latency
  - ms latency from users and devices to cloud data centers
  - How to address? Computing continuum

- Cloud portability
  - Customer's ability to move and adapt their applications and data between their own systems and cloud services, or across different cloud providers and deployment models
  - Risks of vendor lock-in
    - E.g., specific features offered by PaaS provider make it difficult to switch to another provider
    - E.g., difficulty in extracting data and migrating it to different provider
  - Towards portability
    - Containers: enable applications to run consistently across different cloud environments
    - Cloud automation tools (IaC tools)

Issues for cloud customers

- Cloud interoperability
  - Ability of public cloud services, private cloud services, and cloud service customer's system to understand and work with each other (compatibility of interfaces, configuration, authentication and authorization, methods data formats, ...)
  - Standards for cloud portability and interoperability
    - ISO/IEC 19941:2017, IEEE 2302-2021 Standard for Intercloud Interoperability and Federation
  - European initiative GAIA-X https://gaia-x.eu

- SLA negotiation and management
  - Lack of SLIs based on end-user experience
  - SLA monitoring often on customers' shoulders

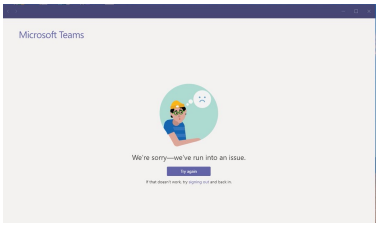
Issues for cloud customers

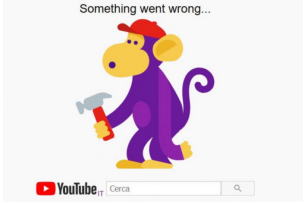
- The two faces of scalability and elasticity
  - Can your application scale?
    - Design application to leverage cloud capabilities
  - Does cloud provider support real elasticity?
    - Automatically provision and de-provision resources on demand, without performance degradation
    - Cloud providers impose limits on resources (e.g., number of VMs, storage size)
    - Stateful applications can face challenges with elasticity because maintaining session persistence and data consistency can be complex

Jiang et al., The Limit of Horizontal Scaling in Public Clouds, ACM TOMPSEC, 2020
https://media.qyjohn.net/blog/2020_Horizontal_Scaling.pdf

Issues for cloud customers

- Cloud outages: examples
  - 2020: AWS Kinesis went down, impacting other AWS services that use it (cascading failure). Cause: one connection per OS thread design led to resource exhaustion https://aws.amazon.com/message/11201
  - 2023: Google Cloud services in Europe hit by outage when Paris' data center caught fire https://thenewstack.io/google-cloud-services-hit-by-outage-in-paris
  - 2024: Microsoft cloud services, including Azure and Teams, experienced a global disruption due to a defective update from CrowdStrike, a cybersecurity company https://tinyurl.com/4tt3ex6p
  - 2025: Google Cloud outage affected Gmail, Drive, Meet, and third-party platforms like Spotify and Discord. Cause: technical failure in Google Cloud's API management system, leading to invalid quota updates https://tinyurl.com/4v7musa8
  - More examples in 2025 https://tinyurl.com/3pveavn6





Issues for cloud providers

- Increasing energy consumption
  - Energy consumption of data centers: 200 TWh in 2016, 460 TWh in 2022, 1000 TWh in 2026
- Uncertainty in service demands
  - Example of provider-side solution: spot instance market to sell unused Amazon EC2 capacity
- SLA management
  - Cascading effect
- Cloud interoperability
  - Interoperable tools with standardized interfaces
- Lack of skilled expertise

Summing up

- Cloud computing is:
  - here to stay and one of the most hyped subjects in IT
  - increasingly becoming an integral concept in IT
  - major trend in the way digital services are designed, implemented and delivered
  - key factor for the digitalization of the whole society
- However, cloud and its evolutions are complex systems
  - Heterogeneous resources, many technologies, plethora of applications with heterogeneous workloads and complex interactions
  - All difficult aspects of large-scale distributed systems
  - Recall: "old" problems become new problems on a totally different scale and open to new solutions